IMT Mines Alès
École Mines-Télécom

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

The Agentic Paradigm

Module 2IAiasd 9.7 – LLM to Agent: Why, What and How

13/10/2026

Komlan AYITE

2ia

Outline

1. Module Overview
1.1. What this module is about
1.2. Roadmap for today

2. From LLM to Agent
2.1. The fundamental shift
2.2. Formal definition

3. Components of an Agent
3.1. LLM, Tools, Memory, Orchestrator

4. The Four Agentic Patterns
4.1. Reflection, Tool Use, Planning, Multi-Agent
4.2. ReAct in detail

5. Local Models & Preview
5.1. Ollama and qwen2.5:3b
5.2. Preview of CM2 + TD1

Module Overview

Welcome to 2IAiasd 9.7 : Agentic AI

You already know...

- Transformer architecture (from your other courses)
- How an LLM generates text token by token
- Python, deep learning basics

The central question of this module

An LLM can *answer* questions.

But can it *solve* problems?
Can it *act* in the world?

That is what an AI agent does.

30 hours — what you will build

- **CM1+TD1** (today) : The agentic paradigm
- **CM2+TP1** (Oct 16) : First agent: tools, prompts, ReAct
- **TP2** (Nov) : RAG pipeline on real documents
- **TP3+TP4** (Nov) : Multi-agents + LangGraph
- **EDT1–4** (Nov–Dec) : Your own agent project
- **Defence** (Dec) : Demo + evaluation

No API key needed, we use local models.

Roadmap for Today

I. From LLM to Agent

- What changes when an LLM acts
- The perception-action loop
- Three levels of autonomy

II. Components of an Agent

- LLM, Tools, Memory, Orchestrator
- Short-term vs long-term memory
- What a tool really is

III. The Four Agentic Patterns

- Reflection, Tool Use, Planning, Multi-Agent
- How they structure this module

IV. Local Models & What's Next

- Why we run LLMs locally (Ollama)
- qwen2.5:3b for tool calling
- Preview of CM2 + TD1

Outcome of today

After this lecture you will understand *what* you are building in TD1, and *why* each step matters.

From LLM to Agent

An LLM Answers. An Agent Acts.

A language model (what you know)

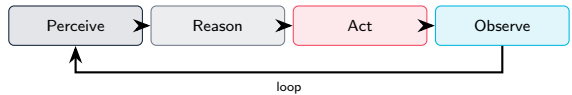
- Input: text. Output: text.
- **Stateless** — no memory between calls
- **Passive** — responds to what it is given
- **One shot** — one prompt → one answer

Prompt ⟶ LLM ⟶ Text

(one round trip, then stop)

An AI agent (what we build)

- Input: goal. Output: result in the world.
- **Stateful** — remembers what it has done
- **Active** — decides what to do next
- **Iterative** — loops until goal is reached



The key insight

The LLM is still the **brain**. What changes is that it now has **hands** (tools), **memory** (state), and a **loop** that keeps it working until it is done.

A Concrete Example: Research Assistant

LLM only

"Summarise the latest advances in agentic AI."

- Answers from training data (may be outdated)
- Cannot verify facts
- One answer, then stops

Result: plausible text, possibly hallucinated

Agent with tools

"Summarise the latest advances in agentic AI."

1. *Thinks:* I need fresh sources
2. *Acts:* calls `web_search("agentic AI 2026")`
3. *Observes:* 5 results returned
4. *Acts:* calls `fetch_page(url)`
5. *Thinks:* I have enough, I can summarise
6. *Answers:* grounded, sourced summary

Result: accurate, verifiable, up-to-date

This is TD1, Exercise 5 in miniature

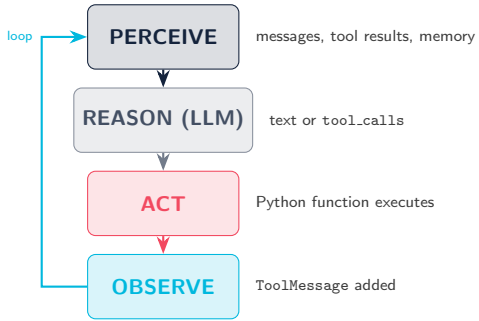
You will build exactly this loop with math tools instead of web search. The structure is **identical**: tools, loop, observe, stop when done.

Formal Definition of an AI Agent

Definition (Russell & Norvig, 2020)

An agent is anything that **perceives** its environment through sensors and **acts** upon it through actuators.

An **LLM-based agent** uses a language model as its reasoning engine.



The four elements (you code all four in TD1):

- **Perceive** — read messages + memory + tool results
- **Reason** — LLM decides what to do next
- **Act** — call a tool, write a file, send a message
- **Observe** — receive result, update state

Three Levels of Autonomy

Stateless Chatbot

- No memory between turns
- One prompt → one answer
- No tools, no loop

Ex: raw API call with no history

AI Agent (what we build)

- Short + long-term memory
- Multi-step ReAct loop
- Tool use + human-in-the-loop
- Supervised goal

Ex: TD1 ReAct loop LangGraph project

Autonomous System

- Episodic + semantic memory
- Self-sets sub-goals
- Multi-agent delegation
- Minimal human oversight

Ex: Devin, AutoGPT

We build agents, not autonomous systems

Autonomy compounds errors. In 2026, **human-in-the-loop** is the production standard. You will implement an interruption checkpoint in TP4.

Components of an Agent

Component 1 : The LLM (The Brain)

What the LLM does in an agent

- Reads the full message history (perception)
- Decides: *"Call a tool, or answer directly?"*
- If tool: returns `tool_calls` with name + args
- If done: returns plain text (the final answer)

The LLM never executes anything.

It only *declares intent*.

Your code runs the tool and feeds the result back.

In TD1, Exercise 5

`response.tool_calls` contains the list of calls the model wants to make. You execute them and append a `ToolMessage`.

What the LLM actually receives

```
[SystemMessage]
  "You are a helpful assistant."

[HumanMessage]
  "What is (3+5) * 12?"

[AIMessage + tool_calls]
  {name: "add", args: {a:3, b:5}}

[ToolMessage]
  "8"

[AIMessage + tool_calls]
  {name: "multiply", args: {a:8, b:12}}

[ToolMessage]
  "96"

[AIMessage]
  "The answer is 96." ← done
```

Component 2 : Tools (The Hands)

What is a tool?

- Any Python function the LLM is allowed to call
- Described by a **JSON schema** (name + description + parameters)
- The schema is injected into the system prompt automatically
- The docstring is your prompt *to the model* about the tool

Rule: the docstring IS the tool's prompt

Good: "Divide a by b. Raises an error if b is zero."
Bad: "Does division."

A vague docstring → wrong tool calls.
A precise docstring → reliable tool calls.

Tools in TD1 : Exercise 4

- `get_word_count(text)`
- `celsius_to_fahrenheit(c)`
- `get_current_date()`
- `calculate_bmi(weight, height)`

Tools in TD1 : Exercise 5 (ReAct agent)

- `add(a, b)`
- `subtract(a, b)`
- `multiply(a, b)`
- `divide(a, b)`
- `power(base, exp)`

The agent chains these to solve multi-step maths.

Component 3 : Memory (The State)

Short-term memory : in the context window

- The `messages` list is the agent's working memory
- Every turn: append Human, AI, and Tool messages
- The LLM "remembers" by re-reading this list
- Limit: the context window (32k–1M tokens)
- **In TD1:** your messages list grows at each ReAct step

Long-term memory : external storage

- Vector database (ChromaDB, FAISS, Pinecone)
- Retrieved on demand via semantic search
- Persists between sessions
- **You build this in TP2** (RAG pipeline)

TD1 Exercise 5 : messages list evolution

```
messages = [SystemMessage, HumanMessage]

--- step 1 ---
append AIMessage (tool_calls=[add])
append ToolMessage (content=''8'')

--- step 2 ---
append AIMessage (tool_calls=[multiply])
append ToolMessage (content=''96'')

--- step 3 (no more tool_calls) ---
append AIMessage (content=''96'')

→ loop exits, return final answer
```

Component 4 : The Orchestrator (The Loop)

The orchestrator is just a `while` loop

- Call the LLM with current messages
- If `response.tool_calls` not empty: execute tools, append results, loop again
- If `response.tool_calls` empty: return the content — done

You write this loop in TD1 : Exercise 5

```
while step < max_steps:
  response = llm.invoke(messages)
  if not response.tool_calls:  break
  for tc in response.tool_calls:
    result = tools_map[tc[''name'']]
          .invoke(tc[''args''])
    messages.append(ToolMessage...)
return response.content
```

In Thursday's CM2 you will see

How this loop maps to **invoke / stream / batch** — the three ways to call an LLM in LangChain.

And how **LangGraph** (TP3 - 4) replaces this manual loop with a proper state machine.

`max_steps` is your safety net

Always set a maximum number of iterations. Without it, a confused model can loop forever.

In production: **10 steps** is a good default.

The Four Agentic Patterns

The Four Patterns (Andrew Ng, 2024)

1. Reflection

The agent critiques its own output and rewrites until satisfied.

Generate → critique → improve

When: writing, code generation, report drafting

2. Tool Use

The agent calls external functions to act on the world.

This is the core of TD1

When: any task needing real data or real actions

3. Planning

The agent decomposes a complex task into ordered sub-steps.

ReAct, Tree of Thoughts, plan-and-execute

When: multi-step research, complex workflows

4. Multi-Agent

Specialised agents collaborate, each handling one sub-task.

CrewAI, AutoGen, LangGraph

When: large tasks needing diverse expertise

Module roadmap through the four patterns

CM1+CM2+TD1 → **Tool Use** (ReAct loop) | TP2 → **Tool Use** (RAG as a retrieval tool) | TP3 → **Multi-Agent** (CrewAI) | TP4 → **Planning** (LangGraph) | Project → all four combined

Pattern 2 in Detail — Tool Use & ReAct

ReAct = Reason + Act

The model alternates between:

- **Thought** — reasoning step (internal monologue)
- **Action** — tool call with arguments
- **Observation** — tool result fed back into context

Example trace (TD1, Exercise 5)

```
Thought:  I need to add 3 and 5 first.
Action:  add(a=3, b=5)
Observation:  8

Thought:  Now multiply 8 by 12.
Action:  multiply(a=8, b=12)
Observation:  96

Answer:  The result is 96.
```

Why ReAct works

- Forces the model to externalise reasoning
- Each tool result corrects the model's knowledge
- Loop stops naturally when no tool is needed
- Same pattern scales to RAG, web search, code execution

ReAct is the foundation of TD1

Exercise 4: define tools + single tool call
Exercise 5: full ReAct loop

In TP3–4 you will replace the manual loop with **LangGraph** — a proper state machine that manages this automatically.

Local Models & Preview

Why We Run LLMs Locally

The practical reality of this course

- API keys = money + account creation + internet dependency
- Free tiers sometimes refuse tool calls
- GDPR: data sent to OpenAI = legal friction in production

The solution: Ollama

- Runs quantised LLMs locally (GGUF format)
- Exposes the same REST API as OpenAI
- LangChain: swap `ChatOpenAI` for `ChatOllama` — nothing else changes
- Works offline once pulled

qwen2.5:3b — our model for TD1

- 3 billion parameters (Alibaba, 2024)
- **Native tool calling** support ✓
- Runs on 6 GB RAM (CPU only)
- Fast enough for interactive TD sessions
- Apache-2.0 licence — fully open-source

Same interface as GPT-4o

```
# OpenAI (needs API key)
from langchain_openai import ChatOpenAI
llm = ChatOpenAI(model=''gpt-4o'')

# Ollama (no key, runs locally)
from langchain_ollama import ChatOllama
llm = ChatOllama(model=''qwen2.5:3b'')

# Everything else is IDENTICAL
```

Preview: CM2 + TD1 on Thursday

CM2 (1h) will give you the code vocabulary

- `invoke` / `stream` / `batch`
  — three ways to call an LLM
- `SystemMessage`, `HumanMessage`, `AIMessage`
  — the message roles
- Prompting: zero-shot, few-shot, chain-of-thought
- `with_structured_output` + Pydantic
- `@tool`, `bind_tools` — attaching tools

TD1 (1h) : what you build step by step

Ex 0 Setup Ollama + LangSmith (optional)

Ex 1 `invoke` / `stream` / `batch`

Ex 2 Messages & system prompts

Ex 3 Structured output with Pydantic

Ex 4 Define tools + single tool call

Ex 5 **Full ReAct agent** — the loop from today

The thread from CM1 to TD1

CM1 taught you *why* the loop works.
CM2 gives you the *vocabulary*.
TD1 makes you *build* it yourself.

Summary: 5 Things to Remember

1. **LLM ≠ Agent**
   An LLM answers. An agent acts in a loop until its goal is reached.
2. **Four components**
   LLM (brain) · Tools (hands) · Memory (state) · Orchestrator (loop). You code all four in TD1.
3. **Memory = messages list**
   Short-term is the growing list of messages. Long-term = vector DB (TP2).
4. **ReAct = the core pattern**
   Reason + Act. The LLM declares tool calls, you execute them, results go back into messages. Loop stops when no tool call is returned.
5. **Local models = no API key**
   `qwen2.5:3b` via Ollama supports native tool calling, runs on 6 GB RAM, same LangChain interface as GPT-4o.

For Thursday

```
ollama pull qwen2.5:3b
pip install langchain-ollama langsmith
pydantic
```

IMT Mines Alès
École Mines-Télécom

RÉPUBLIQUE FRANÇAISE
Liberté
Égalité
Fraternité

Thanks!

Komlan AYITE
komlan.ayite@mines-ales.fr
IMT Mines Alès / CERIS — Dept. 2IA

2ia